

Discrete Mathematics¹
<http://lcs.ios.ac.cn/~znj/DM2017>

Naijun Zhan

April 5, 2017

¹Special thanks to Profs Hanpin Wang (PKU) and Lijun Zhang (ISCAS) for their courtesy of the slides on this course.

Contents

1. **The Foundations: Logic and Proofs**
2. **Basic Structures: Sets, Functions, Sequences, Sums, and Matrices**
3. **Algorithms**
4. Number Theory and Cryptography
5. **Induction and Recursion**
6. Counting
7. Discrete Probability
8. Advanced Counting Techniques
9. Relations
10. Graphs
11. Trees
12. Boolean Algebra
13. Modeling Computation

Chapter 1

The Foundations: Logic and Proofs

Logic in Computer Science

During the past fifty years there has been extensive, continuous, and growing interaction between logic and computer science. In many respects, logic provides computer science with both a unifying foundational framework and a tool for modeling computational systems. In fact, logic has been called the calculus of computer science. The argument is that logic plays a fundamental role in computer science, similar to that played by calculus in the physical sciences and traditional engineering disciplines. Indeed, logic plays an important role in areas of computer science as disparate as machine architecture, computer-aided design, programming languages, databases, artificial intelligence, algorithms, and computability and complexity. **Moshe Vardi**

- The origins of logic can be dated back to Aristotle's time.
- The birth of **mathematical logic**:
 - Leibnitz's idea
 - Russell paradox
 - Hilbert's plan
 - Three schools of modern logic:
 - logicism (Frege, Russell, Whitehead)
 - formalism (Hilbert)
 - intuitionism (Brouwer)
- One of the central problem for logicians is that: "why is this proof correct/incorrect?"
- Boolean algebra owes to George Boole.
- Now, we are interested in: "is the program correct?"

1.1 Propositional Logic – An Appetizer

Definition 1.1.1 (Proposition). *A proposition is a declarative sentence that is either true or false, but not both.*

Definition 1.1.2 (Propositional Logic). *Fix a countable proposition set AP . Syntax of propositional formulas in BNF (Backus-Naur form) is given by:*

$$\varphi ::= p \in AP \mid \neg\varphi \mid \varphi \wedge \varphi$$

Accordingly,

- *Atomic proposition $p \in AP$ is a formula.*
- *Compound formulas: $\neg\varphi$ (negation) and $\varphi \wedge \psi$ (conjunction), provided that φ and ψ are formulas.*

For $p \in AP$, negation and conjunction, we can construct the truth tables. We define the following derived operators:

- Disjunction: $\varphi \vee \psi := \neg(\neg\varphi \wedge \neg\psi)$
- Implication: $\varphi \rightarrow \psi := \neg\varphi \vee \psi$
- Bi-implication: $\varphi \leftrightarrow \psi := (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$
- Exclusive Or: $\varphi \oplus \psi := (\varphi \vee \psi) \wedge (\neg(\varphi \wedge \psi))$

Definition 1.1.3 (Precedence of Logical Operators). *Operators $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$ have precedence 1, 2, 3, 4, 5, respectively.*

Definition 1.1.4 (Logic and Bit Operators).

- *A bit is a symbol with possible values 0 and 1. A Boolean variable is a variable with value true or false.*
- *Computer bit operations correspond to logic connectives: OR, AND, XOR in various programming languages correspond to $\vee$, $\wedge$, $\oplus$, respectively.*
- *A bit string is a sequence of zero or more bits. The length of this string is the number of bits in the string.*
- *Bitwise OR, bitwise AND and bitwise XOR of two strings of the same length are the strings that have as their bits the OR, AND and XOR of the corresponding bits in the two strings, respectively.*

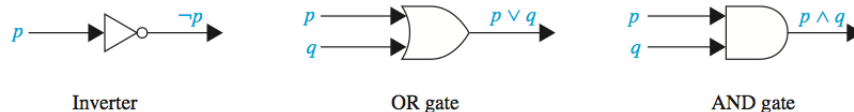
1.2 Applications of Propositional Logic

Definition 1.2.1 (Applications).

- *System Specifications: The automated reply cannot be sent when the file system is full.*
- *Boolean Searches: (one | two) - (three)*
- *Logic Puzzles. Knights always tell the truth, and the opposite knaves always lie. A says: "B is a knight". B says "The two of us are opposite types"*

Definition 1.2.2 (Logic Circuit).

- *Propositional logic can be applied to the design of computer hardware.*
- Claude Shannon**
- *A logic circuit receives input signals $p_1, p_2, \dots, p_n$ and produces an output s . Complicated digital circuits are constructed from three basic circuits, called gates.*



- *Build a digital circuit producing $(p \vee \neg r) \wedge (\neg p \vee (q \vee \neg r))$.*

1.3 Propositional Equivalences

A formula φ is called a

- *tautology* if it is always true, no matter what the truth values of the propositional variables are;
- *contradiction* if it is always false;
- *contingency* if it is neither a tautology nor a contradiction.

Moreover, φ is *satisfiable* if it is either a tautology or a contingency, *unsatisfiable* if it is a contradiction.

Formulas φ and ψ are called *logically equivalent* if $\varphi \leftrightarrow \psi$ is a tautology. This is denoted by $\varphi \equiv \psi$.

Definition 1.3.1 (Logical Equivalence). *Show the following logical equivalences:*

1. *Identity laws:*

$$\varphi \wedge \mathbf{T} \equiv \varphi, \varphi \vee \mathbf{F} \equiv \varphi$$

2. *Dominations laws*

$$\varphi \vee \mathbf{T} \equiv \mathbf{T}, \varphi \wedge \mathbf{F} \equiv \mathbf{F}$$

3. *Idempotent laws*

$$\varphi \vee \varphi \equiv \varphi, \varphi \wedge \varphi \equiv \varphi$$

4. *Double negation law*

$$\neg(\neg\varphi) \equiv \varphi$$

5. *Commutative laws*

$$\varphi \vee \psi \equiv \psi \vee \varphi, \varphi \wedge \psi \equiv \psi \wedge \varphi$$

6. *Associative laws*

$$(\varphi_1 \vee \varphi_2) \vee \varphi_3 \equiv \varphi_1 \vee (\varphi_2 \vee \varphi_3), (\varphi_1 \wedge \varphi_2) \wedge \varphi_3 \equiv \varphi_1 \wedge (\varphi_2 \wedge \varphi_3)$$

7. *Distributive laws*

$$\varphi_1 \vee (\varphi_2 \wedge \varphi_3) \equiv (\varphi_1 \vee \varphi_2) \wedge (\varphi_1 \vee \varphi_3),$$

$$\varphi_1 \wedge (\varphi_2 \vee \varphi_3) \equiv (\varphi_1 \wedge \varphi_2) \vee (\varphi_1 \wedge \varphi_3)$$

8. *De Morgan's laws*

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi, \neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$$

9. *Absorption laws*

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi, \varphi \wedge (\varphi \vee \psi) \equiv \varphi$$

10. *Negation laws*

$$\varphi \vee \neg\varphi \equiv \mathbf{T}, \varphi \wedge \neg\varphi \equiv \mathbf{F}$$

Definition 1.3.2 (Logical Equivalence). *Logical equivalences involving conditional statements:*

1. $\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi$
2. $\varphi \rightarrow \psi \equiv \neg\psi \rightarrow \neg\varphi$
3. $\varphi \vee \psi \equiv \neg\varphi \rightarrow \psi$
4. $\varphi \wedge \psi \equiv \neg(\varphi \rightarrow \neg\psi)$
5. $\varphi \wedge \neg\psi \equiv \neg(\varphi \rightarrow \psi)$
6. $(\varphi_1 \rightarrow \varphi_2) \wedge (\varphi_1 \rightarrow \varphi_3) \equiv \varphi_1 \rightarrow (\varphi_2 \wedge \varphi_3)$
7. $(\varphi_1 \rightarrow \varphi_3) \wedge (\varphi_2 \rightarrow \varphi_3) \equiv (\varphi_1 \vee \varphi_2) \rightarrow \varphi_3$
8. $(\varphi_1 \rightarrow \varphi_2) \vee (\varphi_1 \rightarrow \varphi_3) \equiv \varphi_1 \rightarrow (\varphi_2 \vee \varphi_3)$
9. $(\varphi_1 \rightarrow \varphi_3) \vee (\varphi_2 \rightarrow \varphi_3) \equiv (\varphi_1 \wedge \varphi_2) \rightarrow \varphi_3$
10. $\varphi \leftrightarrow \psi \equiv \neg\varphi \leftrightarrow \neg\psi$
11. $\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi)$
12. $\neg(\varphi \leftrightarrow \psi) \equiv \varphi \leftrightarrow \neg\psi$

1.4 Induction and Recursion

PRINCIPLE OF MATHEMATICAL INDUCTION To prove that $P(n)$ is true for all positive integers n , where $P(n)$ is a propositional function, we complete two steps:

- **BASIS STEP:** We verify that $P(1)$ is true.
- **INDUCTIVE STEP:** We show that the conditional statement $P(k) \rightarrow P(k + 1)$ is true for all positive integers k .

Expressed as a rule of inference for *first-order logic*, this proof technique can be stated as:

$$\Phi := (P(1) \wedge \forall k.(P(k) \rightarrow P(k + 1))) \rightarrow \forall n.P(n)$$

Exercise 1.4.1. Prove $1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$.

STRONG INDUCTION (Second principle of mathematical induction) To prove that $P(n)$ is true for all positive integers n , where $P(n)$ is a propositional function, we complete two steps:

- BASIS STEP: We verify that the proposition $P(1)$ is true.
- INDUCTIVE STEP: We show that the conditional statement $(P(1) \wedge P(2) \wedge \dots \wedge P(k)) \rightarrow P(k+1)$ is true for all positive integers k .

Expressed as a rule of inference for *first-order logic*, this proof technique can be stated as:

$$\Psi := (P(1) \wedge \forall k. (\bigwedge_{i=1}^k P(i) \rightarrow P(k+1))) \rightarrow \forall n. P(n)$$

Exercise 1.4.2. *Prove that if n is a natural number greater than 1, then n can be written as the product of primes.*

Recursively Defined Sets and Structures and Structural Induction

Definition 1.4.3 (Strings). *The set Σ^* of strings over the alphabet Σ is defined recursively by*

- *BASIS STEP: $\lambda \in \Sigma^*$ (where λ is the empty string containing no symbols).*
- *RECURSIVE STEP: If $w \in \Sigma^*$ and $x \in \Sigma$, then $wx \in \Sigma^*$.*

Definition 1.4.4. *We define the set of well-formed formulas in propositional logic, denoted by L , from alphabet $\Sigma := AP \cup \{\neg, \rightarrow, (,)\}$.*

- *BASIS STEP: each $p \in AP$ is a well-formed formula.*
- *RECURSIVE STEP: If φ and ψ are well-formed formulas, i.e., $\varphi, \psi \in L$, then $(\neg\varphi)$, $(\varphi \rightarrow \psi)$ are well-formed formulas.*

Thus, the set of well-formed formulas is a subset of $L \subseteq \Sigma^$.*

STRUCTURAL INDUCTION A proof by structural induction consists of two parts.

- **BASIS STEP:** Show that the result holds for all elements specified in the basis step.
- **RECURSIVE STEP:** Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

Remark: The validity of structural induction follows from the principle of mathematical induction for the nonnegative integers.

Exercise 1.4.5. *Show that every well-formed formula for compound propositions contains an equal number of left and right parentheses.*

1.5 Normal Forms

Negation Normal Form

- **Literal:** An atomic proposition p or its negation $\neg p$;
- **Negation Normal Form (NNF):** A formula built up with “ $\wedge$ ”, “ $\vee$ ”, and literals.
- Using repeated DeMorgan and Double Negation, we can transform any formula into a formula with Negation Normal Form.
- **Example:**

$$\begin{aligned}\neg((A \vee B) \wedge \neg C) &\leftrightarrow \text{(DeMorgan)} \\ \neg(A \vee B) \vee \neg\neg C &\leftrightarrow \text{(Double Neg, DeMorgan)} \\ (\neg A \wedge \neg B) \vee C\end{aligned}$$

Disjunction Normal Form

- **Disjunction Normal Form (DNF):** A generalized disjunction of generalized conjunctions of literals.
- Using repeated distribution of $\wedge$ over $\vee$, any NNF formula can be rewritten in DNF (exercise).
- **Example:**

$$\begin{aligned}(A \vee B) \wedge (C \vee D) &\leftrightarrow \text{(Distribution)} \\ [(A \vee B) \wedge C] \vee [(A \vee B) \wedge D] &\leftrightarrow \text{(Distribution)} \\ (A \wedge C) \vee (B \wedge C) \vee (A \wedge D) \vee (B \wedge D)\end{aligned}$$

Conjunction Normal Form

- **Conjunction Normal Form (CNF):** A generalized conjunction of generalized disjunctions of literals.
- Using repeated distribution of $\vee$ over $\wedge$, any NNF formula can be rewritten in CNF (exercise).
- **Example:**

$$\begin{aligned}
 (A \wedge B) \vee (C \wedge D) & \leftrightarrow \text{(Distribution)} \\
 [(A \wedge B) \vee C] \wedge [(A \wedge B) \vee D] & \leftrightarrow \text{(Distribution)} \\
 (A \vee C) \wedge (B \vee C) \wedge (A \vee D) \wedge (B \vee D)
 \end{aligned}$$

Truth table

Unary Connectives

- What other unary connectives are there besides ' $\neg$ '?
- Thinking about this in terms of truth tables, we see that there are 4 different unary connectives:

P	*P
T	T
F	T

P	*P
T	T
F	F

P	*P
T	F
F	T

P	*P
T	F
F	F

Binary Connectives

- The truth table below shows that there are $2^4 = 16$ binary connectives:

P	Q	P*Q
T	T	T/F
T	F	T/F
F	T	T/F
F	F	T/F

In general:
 n sentences $\Rightarrow$

2^n truth value combinations
(i.e. 2^n rows in truth table) $\Rightarrow$

2^{2^n} different n -ary connectives!

Expressive completeness

- What are the truth tables for $(p \wedge q) \vee r$ and $p \wedge (q \vee r)$?
- Truth table for n -ary Boolean function.
- A set of logical connectives is called **functionally complete** if any n -ary Boolean function is definable with it, e.g. $\{\neg, \wedge\}$, $\{\neg, \vee\}$.
- How about $\{\neg, \rightarrow\}$?

Exercise. Exercise 49 of page 16, Exercise 15 of page 23, Exercise 39 of page 35, Exercise 45 of page 36.

1.6 Propositional Logic and Deduction Systems: a Sound and Complete Axiomatization

This section considers a complete axiomatization system such that, a formula is a tautology if and only if it can be derived by means of the axioms and the deduction rules of the system.

1.6.1 Syntax and Semantics of Propositional Logic

Fix a countable proposition set AP , then [formulas](#) of propositional logic are defined by:

Definition 1.6.1 (Syntax). *Syntax of propositional formulas in BNF (Backus-Naur form) is given by:*

$$\varphi ::= p \in AP \mid \neg\varphi \mid \varphi \rightarrow \varphi$$

It generates recursively the set of well-formed formulas, denoted by L :

- Atomic formula: $p \in AP$ implies $p \in L$.
- Compound formulas: $(\neg\varphi)$ and $(\varphi \rightarrow \psi)$, provided that $\varphi, \psi \in L$.

We omit parentheses if it is clear from the context.

Semantics of a formula φ is given w.r.t. an **assignment** $\sigma \in 2^{AP}$, which is a subset of AP . Intuitively, it assigns **true** (or, **T**) to propositions belonging to it, and assigns **false** (or, **F**) to others. Thus, it can also be viewed as a function from AP to $\{\mathbf{T}, \mathbf{F}\}$.

Definition 1.6.2 (Semantics). *Inductively, we may define the relation $\Vdash \subseteq 2^{AP} \times L$ as follows:*

- $\sigma \Vdash p$ iff $p \in \sigma$.
- $\sigma \Vdash \neg\varphi$ iff not $\sigma \Vdash \varphi$ (denoted by $\sigma \nVdash \varphi$).
- $\sigma \Vdash \varphi \rightarrow \psi$ iff either $\sigma \nVdash \varphi$ or $\sigma \Vdash \psi$.

where $(\sigma, \varphi) \in \Vdash$ is denoted as $\sigma \Vdash \varphi$.

The formula φ is called a *tautology* if $\sigma \Vdash \varphi$ for all assignment, it is *satisfiable* if $\sigma \Vdash \varphi$ for some assignment.

1.6.2 The Axiom System: the Hilbert's System

Definition 1.6.3 (Axioms). 1. $\varphi \rightarrow (\psi \rightarrow \varphi)$.

2. $(\varphi \rightarrow (\psi \rightarrow \eta)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \eta))$.

$$3. (\neg\varphi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \varphi).$$

Definition 1.6.4 (MP Rule). 1. $\frac{\varphi \rightarrow \psi \quad \varphi}{\psi}$

Given a formula set Γ , a **deductive sequence** of φ from Γ is a sequence

$$\varphi_0, \varphi_1, \dots, \varphi_n = \varphi$$

where each φ_i should be one of the following cases:

1. $\varphi_i \in \Gamma$.
2. φ_i is an instance of some axiom.
3. There exists some $j, k < i$, such that $\varphi_k = \varphi_j \rightarrow \varphi_i$.

And, we denote by $\Gamma \vdash \varphi$ if there exists such deductive sequence. We write $\Gamma, \psi \vdash \varphi$ for $\Gamma \cup \{\psi\} \vdash \varphi$.

1.6.3 The Axiom System: Soundness

For a formula set Γ and an assignment σ , the *satisfaction relation* $\models$ is defined by: $\sigma \models \Gamma$ iff $\sigma \models \varphi$ for every $\varphi \in \Gamma$.

Observe $\sigma \models \emptyset$ always holds. We say φ is a **logical consequent** of Γ , denoted as $\Gamma \models \varphi$, if $\sigma \models \Gamma$ implies $\sigma \models \varphi$ for each assignment σ .

Thus, φ is a tautology if φ is the logical consequent of $\emptyset$, denoted as $\models \varphi$.

Theorem 1.6.5 (Soundness). *With Hilbert's axiom system, we have that $\Gamma \vdash \varphi$ implies $\Gamma \models \varphi$.*

Proof. By induction of the length of deductive sequence of $\Gamma \vdash \varphi$. □

Corollary 1.6.6. *If $\vdash \varphi$, then $\models \varphi$.*

1.6.4 The Axiom System: Completeness

With Hilbert's axiom system, we have the following elementary properties:

(Fin) If $\Gamma \vdash \varphi$, then there exists some finite subset Γ' of Γ , such that $\Gamma' \vdash \varphi$.

($\in$) If $\varphi \in \Gamma$, then $\Gamma \vdash \varphi$.

($\in_+$) If $\Gamma \vdash \varphi$ and $\Gamma \subseteq \Gamma'$ then $\Gamma' \vdash \varphi$.

($\overline{\text{MP}}$) If $\Gamma_1 \vdash \varphi$ and $\Gamma_2 \vdash \varphi \rightarrow \psi$, and $\Gamma_1, \Gamma_2 \subseteq \Gamma$, then $\Gamma \vdash \psi$.

1.6.5 The Axiom System: Examples of Theorems

Example 1.6.7. (*Ide*): $\vdash \varphi \rightarrow \varphi$

Solution. 1. $\varphi \rightarrow (\varphi \rightarrow \varphi)$

2. $\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)$

3. $(\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)) \rightarrow ((\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi))$

4. $(\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi)$

5. $\varphi \rightarrow \varphi$

Example 1.6.8. ($\rightarrow_-$): If $\Gamma \vdash \varphi \rightarrow \psi$ then $\Gamma, \varphi \vdash \psi$.

Solution. A simple application of $\overline{\text{MP}}$ and $(\in)$.

Example 1.6.9. ($\rightarrow_+$): If $\Gamma, \varphi \vdash \psi$ then $\Gamma \vdash \varphi \rightarrow \psi$.

Solution. By induction of the deductive sequence of $\Gamma, \varphi \vdash \psi$.

Example 1.6.10. (τ): If $\Gamma \vdash \varphi \rightarrow \psi$ and $\Gamma \vdash \psi \rightarrow \eta$, then $\Gamma \vdash \varphi \rightarrow \eta$.

Solution. By $(\rightarrow_-)$, $(\rightarrow_+)$ and $(\in_+)$.

Example 1.6.11. (*Abs*): $\vdash \neg\varphi \rightarrow (\varphi \rightarrow \psi)$.

Solution. 1. $\vdash \neg\varphi \rightarrow (\neg\psi \rightarrow \neg\varphi)$

2. $\vdash (\neg\psi \rightarrow \neg\varphi) \rightarrow (\varphi \rightarrow \psi)$

3. $\vdash \neg\varphi \rightarrow (\varphi \rightarrow \psi)$

Example 1.6.12. (*Abs'*): $\vdash \varphi \rightarrow (\neg\varphi \rightarrow \psi)$

Example 1.6.13. ($\neg_w$): $\neg\varphi \rightarrow \varphi \vdash \varphi$

Solution. 1. $\neg\varphi \rightarrow \varphi \vdash \neg\varphi \rightarrow \varphi$

2. $\vdash \neg\varphi \rightarrow (\varphi \rightarrow \neg(\neg\varphi \rightarrow \varphi))$
3. $\vdash (\neg\varphi \rightarrow (\varphi \rightarrow \neg(\neg\varphi \rightarrow \varphi))) \rightarrow ((\neg\varphi \rightarrow \varphi) \rightarrow (\neg\varphi \rightarrow \neg(\neg\varphi \rightarrow \varphi)))$
4. $\neg\varphi \rightarrow \varphi \vdash \neg\varphi \rightarrow \neg(\neg\varphi \rightarrow \varphi)$
5. $\vdash (\neg\varphi \rightarrow \neg(\neg\varphi \rightarrow \varphi)) \rightarrow ((\neg\varphi \rightarrow \varphi) \rightarrow \varphi)$
6. $\neg\varphi \rightarrow \varphi \vdash (\neg\varphi \rightarrow \varphi) \rightarrow \varphi$
7. $\neg\varphi \rightarrow \varphi \vdash \varphi$

Example 1.6.14. $(\neg\neg_-)$: $\neg\neg\varphi \vdash \varphi$

Solution. 1. $\vdash \neg\neg\varphi \rightarrow (\neg\varphi \rightarrow \varphi)$

2. $\vdash (\neg\varphi \rightarrow \varphi) \rightarrow \varphi$
3. $\vdash \neg\neg\varphi \rightarrow \varphi$
4. $\neg\neg\varphi \vdash \varphi$

Example 1.6.15. $(\neg_s)$: $\varphi \rightarrow \neg\varphi \vdash \neg\varphi$

Solution. 1. $\neg\neg\varphi \vdash \varphi$

2. $\varphi \rightarrow \neg\varphi \vdash \varphi \rightarrow \neg\varphi$
3. $\varphi \rightarrow \neg\varphi, \neg\neg\varphi \vdash \varphi$
4. $\varphi \rightarrow \neg\varphi \vdash \neg\neg\varphi \rightarrow \neg\varphi$
5. $\vdash (\neg\neg\varphi \rightarrow \neg\varphi) \rightarrow \neg\varphi$
6. $\varphi \rightarrow \neg\varphi \vdash \neg\varphi$

Example 1.6.16. $(\neg\neg_+)$: $\varphi \vdash \neg\neg\varphi$

Solution. 1. $\vdash \varphi \rightarrow (\neg\varphi \rightarrow \neg\neg\varphi)$

2. $\vdash (\neg\varphi \rightarrow \neg\neg\varphi) \rightarrow \neg\neg\varphi$

3. $\vdash \varphi \rightarrow \neg\neg\varphi$

4. $\varphi \vdash \neg\neg\varphi$

Example 1.6.17. (R0) $\varphi \rightarrow \psi \vdash \neg\psi \rightarrow \neg\varphi$

(R1) $\varphi \rightarrow \neg\psi \vdash \psi \rightarrow \neg\varphi$

(R2) $\neg\varphi \rightarrow \psi \vdash \neg\psi \rightarrow \varphi$

(R3) $\neg\varphi \rightarrow \neg\psi \vdash \psi \rightarrow \varphi$

Solution. 1. $\varphi \rightarrow \psi, \neg\neg\varphi \vdash \varphi$

2. $\varphi \rightarrow \psi, \neg\neg\varphi \vdash \psi$

3. $\vdash \psi \rightarrow \neg\neg\psi$

4. $\vdash (\neg\neg\varphi \rightarrow \neg\neg\psi) \rightarrow (\neg\psi \rightarrow \neg\varphi)$

5. $\varphi \rightarrow \psi \vdash \neg\psi \rightarrow \neg\varphi$

Definition 1.6.18 (Consistency). *We say a formula set Γ is **consistent**, iff there is some φ such that $\Gamma \not\vdash \varphi$. Moreover, we say φ is **consistent** w.r.t. Γ iff $\Gamma \cup \{\varphi\}$ is consistent.*

Note that we have the theorem $\neg\varphi, \varphi \vdash \psi$ and hence, Γ is

consistent iff for each φ , either $\Gamma \not\vdash \varphi$ or $\Gamma \not\vdash \neg\varphi$.

Further, φ is consistent w.r.t. Γ iff $\Gamma \not\vdash \neg\varphi$. Suppose that $\Gamma \not\vdash \neg\varphi$ and $\Gamma \cup \{\varphi\}$ is inconsistent, then we have $\Gamma, \varphi \vdash \neg\varphi$ hence $\Gamma \vdash \varphi \rightarrow \neg\varphi$. Recall that we have $\varphi \rightarrow \neg\varphi \vdash \neg\varphi$, and this implies $\Gamma \vdash \neg\varphi$, contradiction!

Lemma 1.6.19. *If the formula set Γ is inconsistent, then it has some finite inconsistent subset Δ .*

Theorem 1.6.20. *Γ is consistent iff Γ is satisfiable.*

Proof sketch. The “if” direction is easy: suppose that $\sigma \models \Gamma$ but $\Gamma \vdash \varphi$ and $\Gamma \vdash \neg\varphi$, then $\sigma \models \varphi$ and $\sigma \models \neg\varphi$, contradiction.

For the “only if” direction, let us enumerate all propositional formulas as following (note the cardinality of all such formulas is $\aleph_0$):

$$\varphi_0, \varphi_1, \dots, \varphi_n, \dots$$

Let $\Gamma_0 = \Gamma$ and

$$\Gamma_{i+1} = \begin{cases} \Gamma_i \cup \{\varphi_i\} & \text{if } \Gamma_i \not\vdash \neg\varphi_i \\ \Gamma_i \cup \{\neg\varphi_i\} & \text{otherwise} \end{cases}$$

and finally let $\Gamma^* = \lim_{i \rightarrow \infty} \Gamma_i$.

The formula set Γ^* has the following properties:

1. Each Γ_i is consistent, and Γ^* is also consistent.
2. Γ^* is a **maximal** set, i.e., for each formula φ , either $\varphi \in \Gamma^*$ or $\neg\varphi \in \Gamma^*$.
3. For each formula φ , we have $\Gamma^* \models \varphi$ iff $\varphi \in \Gamma^*$.

Then we have $\sigma \models \Gamma^*$, where $\sigma = \Gamma^* \cap AP$. □

Theorem 1.6.21 (Completeness). *If $\Gamma \models \varphi$, then $\Gamma \vdash \varphi$.*

Proof. Assume by contradiction that $\Gamma \not\models \varphi$, then there is an assignment σ such that $\sigma \models \Gamma \cup \{\neg\varphi\}$. However, this implies that $\sigma \models \Gamma$ and $\sigma \not\models \varphi$, which violates the assumption $\Gamma \models \varphi$. $\square$

Corollary 1.6.22. *$\models \varphi$ implies that $\vdash \varphi$.*

The Axiom System: Compactness

Theorem 1.6.23. *Given a formula set Γ , we have*

1. *Γ is consistent iff each of its finite subsets is consistent;*
2. *Γ is satisfiable iff each of its finite subsets is satisfiable.*

Proof. 1. The first property has been proven (see the previous lemma).

2. With the aforementioned theorem: for propositional logic, a set is satisfiable iff it is consistent.

$\square$

Rules of Inference for Propositional Logic (cf. page 72):

<i>Rule of Inference</i>	<i>Tautology</i>	<i>Name</i>
$\begin{array}{l} p \\ p \rightarrow q \\ \hline \therefore q \end{array}$	$(p \wedge (p \rightarrow q)) \rightarrow q$	Modus ponens
$\begin{array}{l} \neg q \\ p \rightarrow q \\ \hline \therefore \neg p \end{array}$	$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$	Modus tollens
$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$	$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$	Hypothetical syllogism
$\begin{array}{l} p \vee q \\ \neg p \\ \hline \therefore q \end{array}$	$((p \vee q) \wedge \neg p) \rightarrow q$	Disjunctive syllogism
$\begin{array}{l} p \\ \hline \therefore p \vee q \end{array}$	$p \rightarrow (p \vee q)$	Addition
$\begin{array}{l} p \wedge q \\ \hline \therefore p \end{array}$	$(p \wedge q) \rightarrow p$	Simplification
$\begin{array}{l} p \\ q \\ \hline \therefore p \wedge q \end{array}$	$((p) \wedge (q)) \rightarrow (p \wedge q)$	Conjunction
$\begin{array}{l} p \vee q \\ \neg p \vee r \\ \hline \therefore q \vee r \end{array}$	$((p \vee q) \wedge (\neg p \vee r)) \rightarrow (q \vee r)$	Resolution

Exercise 1.6.24. *Show, by applying the rules of the deduction system presented in Section 1.6, the following statements:*

1. $\vdash (\varphi \rightarrow \psi) \rightarrow ((\neg\varphi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \varphi))$
2. $\vdash ((\varphi \rightarrow (\psi \rightarrow \eta)) \rightarrow (\varphi \rightarrow \psi)) \rightarrow ((\varphi \rightarrow (\psi \rightarrow \eta)) \rightarrow (\varphi \rightarrow \eta))$
3. $\vdash (\varphi \rightarrow (\varphi \rightarrow \psi)) \rightarrow (\varphi \rightarrow \psi)$
4. $\vdash \varphi \rightarrow (\psi \rightarrow (\varphi \rightarrow \psi))$
5. $\{\varphi \rightarrow \psi, \neg(\psi \rightarrow \eta) \rightarrow \neg\varphi\} \vdash \varphi \rightarrow \eta$
6. $\varphi \rightarrow (\psi \rightarrow \eta) \vdash \psi \rightarrow (\varphi \rightarrow \eta)$
7. $\vdash ((\varphi \rightarrow \psi) \rightarrow \varphi) \rightarrow \varphi$
8. $\vdash \neg(\varphi \rightarrow \psi) \rightarrow (\psi \rightarrow \varphi)$

Exercise 1.6.25. *Find a deduction showing the correctness of some of the following equivalences, that is, if $\varphi \equiv \psi$, then provide a deduction for $\vdash \varphi \rightarrow \psi$ and for $\vdash \psi \rightarrow \varphi$.*

1. $\varphi \vee (\varphi \wedge \psi) \equiv \varphi$,
2. $(\varphi_1 \rightarrow \varphi_2) \vee (\varphi_1 \rightarrow \varphi_3) \equiv \varphi_1 \rightarrow (\varphi_2 \vee \varphi_3)$.

Exercise 1.6.26 (* not required). *Fill the missing parts of the proofs of the soundness and completeness theorems in Section 1.6.*

Chapter 2

Basic Structures: Sets, Functions, Sequences, Sums, and Matrices

2.1 Sets and Functions

Definition 2.1.1.

- Fix an universal set U . Set operations: union $\cup$, intersection $\cap$, complement $\overline{A}$.
- Set inclusion: $A \subseteq B$ iff for all $a \in A$ it holds $a \in B$.
 $A = B$ iff $A \subseteq B$ and $B \subseteq A$.
- Given a set S , the power set of S is the set of all subsets of the set S . The power set is denoted by $\mathcal{P}(S)$, or 2^S .
- The Cartesian product of sets $A_1, A_2, \dots, A_n$ is defined by:
 $A_1 \times \dots \times A_n := \{(a_1, \dots, a_n) \mid a_i \in A_i \text{ for } i = 1, \dots, n\}$.
- The cardinality of finite set A , denoted by $|A|$, is the number of its elements. The principle of inclusion-exclusion:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

TABLE 1 Set Identities.	
<i>Identity</i>	<i>Name</i>
$A \cap U = A$ $A \cup \emptyset = A$	Identity laws
$A \cup U = U$ $A \cap \emptyset = \emptyset$	Domination laws
$A \cup A = A$ $A \cap A = A$	Idempotent laws
$\overline{\overline{A}} = A$	Complementation law
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Commutative laws
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive laws
$\overline{A \cap B} = \overline{A} \cup \overline{B}$ $\overline{A \cup B} = \overline{A} \cap \overline{B}$	De Morgan's laws
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Absorption laws
$A \cup \overline{A} = U$ $A \cap \overline{A} = \emptyset$	Complement laws

Definition 2.1.2. *Let A and B be nonempty sets. A function $f: A \rightarrow B$ from A to B is an assignment of exactly one element of B to each element of A . We write $f(a) = b$ if $b \in B$ is assigned by f to the element $a \in A$. We say that*

- *A is the domain of f ,*
- *B is the codomain of f .*
- *If $f(a) = b$, we say that b is the image of a and a is a preimage of b .*
- *The range, or image, of f is the set of all images of elements of A .*

Given two sets A and B ,

- a relation R between A and B is a subset of $A \times B$.
- a function from A to B must be a relation, but the inverse does not hold in general.

Definition 2.1.3. *Let A and B be two sets. The function $f: A \rightarrow B$ is called*

- *one-to-one, or an injection, if and only if $f(a) = f(b)$ implies that $a = b$ for all a and b in the domain of f .*

- onto, or a surjection, if and only if for every element $b \in B$ there is an element $a \in A$ with $f(a) = b$.
- one-to-one correspondence, or a bijection, if it is both one-to-one and onto.

Definition 2.1.4. *Let A , B , and C be three sets.*

- *Let $f: A \rightarrow B$ be bijective. The inverse function of f , denoted by f^{-1} , is the function that assigns to an element $b \in B$ the unique element $a \in A$ such that $f(a) = b$.*
- *Let $g: A \rightarrow B$ and let $f: B \rightarrow C$. The composition of the functions f and g , denoted $f \circ g$, is defined by*

$$(f \circ g)(a) = f(g(a))$$

Let A , B , and C be three sets.

- Let $R \subseteq A \times B$, then the *inverse relation* of R , denoted by R^{-1} , is a subset of $B \times A$ such that $(b, a) \in R^{-1}$ iff $(a, b) \in R$, which is a relation between B and A .
- Let $R_1 \subseteq A \times B$ and $R_2 \subseteq B \times C$. The *composition* of the relations R_1 and R_2 , denoted $R_1 \circ R_2$, is defined by

$$(R_1 \circ R_2) = \{(a, c) \mid \exists b. (a, b) \in R_1 \wedge (b, c) \in R_2\}$$

Definition 2.1.5 (Some Notations). *Let A and B be two sets.*

- *For a function $f: A \rightarrow B$, and a set $D \subseteq A$, we use $f|_D: D \rightarrow B$ to denote the function f with domain restricted to the set D .*
- *A partial function f from a set A to a set B is an assignment to each element $a \in D \subseteq A$, called the domain of definition of f , of a unique element $b \in B$. We say that f is undefined for elements in $A \setminus D$. When $D = A$, we say that f is a total function.*

Definition 2.1.6. *Consider the set $U = 2^{AP}$ of all assignments. The semantic bracket is a function $\llbracket \cdot \rrbracket: PL \rightarrow 2^U$ defined by:*

- $\llbracket p \rrbracket = \{\sigma \in U \mid p \in \sigma\},$

- $\llbracket \neg \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket},$
- $\llbracket \varphi \rightarrow \psi \rrbracket = \overline{\llbracket \varphi \rrbracket} \cup \llbracket \psi \rrbracket.$

Is $\llbracket \cdot \rrbracket$ injective, surjective, or bijective?

Definition 2.1.7 (Currying). Let $A_1, \dots, A_n$ and B be sets

- for any $f \in A_1 \times A_2 \times \dots \times A_n \rightarrow B$, there exists $f' \in A_1 \rightarrow (A_2 \rightarrow \dots (A_n \rightarrow B) \dots)$, such that for any $(a_1, \dots, a_n) \in A_1 \times \dots \times A_n$, it follows $f(a_1, \dots, a_n) = f'(a_1)(a_2) \dots (a_n).$
- Conversely, for any $f' \in A_1 \rightarrow (A_2 \rightarrow \dots (A_n \rightarrow B) \dots)$, there exists $f \in A_1 \times A_2 \times \dots \times A_n \rightarrow B$, such that for any $(a_1, \dots, a_n) \in A_1 \times \dots \times A_n$, it follows $f'(a_1)(a_2) \dots (a_n) = f(a_1, \dots, a_n).$

2.2 Cardinality, Diagonalization Argument

Definition 2.2.1. *Let A and B be two sets.*

- *The sets A and B have the same cardinality if and only if there is a one-to-one correspondence from A to B . When A and B have the same cardinality, we write $|A| = |B|$.*
- *If there is a one-to-one function from A to B , the cardinality of A is less than or the same as the cardinality of B and we write $|A| \leq |B|$. Moreover, when $|A| \leq |B|$ and A and B have different cardinality, we say that the cardinality of A is less than the cardinality of B and we write $|A| < |B|$.*

Definition 2.2.2. *A set that is either finite or has the same cardinality as the set of positive integers is called countable. A set that is not countable is called uncountable. When an infinite set S is countable, we denote the cardinality of S by $\aleph_0$. We write $|S| = \aleph_0$ and say that S has cardinality aleph null.*

Theorem 2.2.3 (SCHRÖDER-BERNSTEIN THEOREM). *If A and B are sets with $|A| \leq |B|$ and $|B| \leq |A|$, then $|A| = |B|$.*

Lemma 2.2.4. *Prove that*

1. *The union, intersection of countable sets is countable.*
2. *The set $\mathbb{N}^2$ is countable.*
3. *The set $\mathbb{Z}$ of integer numbers is countable.*
4. *The set $\mathbb{Q}$ of rational numbers is countable.*
5. *The set $\mathbb{N}^c$ with $c \in \mathbb{N}$ is countable.*
6. *The countable union of countable sets is countable.*
7. *The set $\mathbb{N}^*$ is countable.*

Lemma 2.2.5. *Prove that*

1. $|[0, 1]| = |(0, 1]| = |[0, 1)| = |(0, 1)|.$

2. $|(0, 1]| = |[1, \infty)|.$

3. $|[0, 1]| = |[0, k]| = |[0, \infty)| = |\mathbb{R}|.$

4. $|\{0, 1\}^\omega| = |[0, 1]|.$

5. $|2^{\mathbb{N}}| = |\{0, 1\}^\omega|.$

6. $|2^{\mathbb{N}}| = |\{ f \mid f: \mathbb{N} \rightarrow \{0, 1\} \}|.$

Lemma 2.2.6 (Cantor diagonalization argument).

- *The set $\mathbb{R}$ of real numbers is uncountable.*
- *For a set A , it holds: $|A| < |2^A|$.*

Exercise

- Prove $\mathbb{R}$ is uncountable.
- Prove $(\varphi_1 \rightarrow \psi_1) \vee (\varphi_2 \rightarrow \psi_2) \vdash (\varphi_1 \rightarrow \psi_2) \vee (\varphi_2 \rightarrow \psi_1)$.
- Prove $(\varphi \wedge \psi) \rightarrow \theta, \theta \rightarrow \chi, \psi \wedge \neg \chi \vdash \neg \varphi$.

Chapter 3

First Order Logic (FOL)

3.1 Syntax of FOL

Propositional logic is a **coarse language**, which only concerns about propositions and boolean connectives. Practically, this logic is not powerful enough to describe important properties we are interested in.

Example 3.1.1 (Syllogism of Aristotle). *Consider the following assertions:*

1. *All men are mortal.*
2. *Socrates is a man.*
3. *So Socrates would die.*

$$\forall x(Man(x) \rightarrow Mortal(x))$$

Definition 3.1.2. *First order logic is an extension of proposition logic:*

1. *To accept parameters, it generalized **propositions** to **predicates**.*

2. To designate elements in the domain, it is equipped with *functions* and *constants*.
3. It also involves *quantifiers* to capture infinite conjunction and disjunction.

Definition 3.1.3. We are given:

- an *arbitrary* set of *variable symbols* $VS = \{x, y, x_1, \dots\}$;
- an *arbitrary* set (maybe empty) of *function symbols* $FS = \{f, g, f_1, \dots\}$, where each symbol has an *arity*;
- an *arbitrary* set (maybe empty) of *predicate symbols* $PS = \{P, Q, P_1, \dots\}$, where each symbol has an *arity*;
- an equality symbol set ES which is either empty or one element set containing $\{\approx\}$.

Let $L = VS \cup \{(\cdot), \rightarrow, \neg, \forall\} \cup FS \cup PS \cup ES$. Here $VS \cup \{(\cdot), \rightarrow, \neg, \forall\}$ are referred to as logical symbols, and $FS \cup PS \cup ES$ are referred to as non-logical symbols.

We often make use of the

- set of *constant symbols*, denoted by $CS = \{a, b, a_1, \dots\} \subseteq FS$, which consist of function symbols with arity 0;

- set of *propositional symbols*, denoted by $PS = \{p, q, p_1, \dots\} \subseteq FS$, which consist of predicate symbols with arity 0.

Definition 3.1.4 (FOL terms). *The terms of the first order logic are constructed according to the following grammar:*

$$t ::= x \mid ft_1 \dots t_n$$

where $x \in VS$, and $f \in FS$ has arity n .

Accordingly, the set T of terms is the smallest set satisfying the following conditions:

- each variable $x \in VS$ is a term.
- Compound terms: $ft_1 \dots t_n$ is a term (thus in T), provided that f is a n -arity function symbol, and $t_1, \dots, t_n \in T$. Particularly, $a \in CS$ is a term.

We often write $f(t_1, \dots, t_n)$ for the compound terms.

Definition 3.1.5 (FOL formulas). *The well-formed formulas of the first order logic are constructed according to the following grammar:*

$$\varphi ::= Pt_1 \dots t_n \mid \neg\varphi \mid \varphi \rightarrow \varphi \mid \forall x\varphi$$

where $t_1, \dots, t_n$ are terms, $P \in PS$ has arity n , and $x \in VS$.

We often write $P(t_1, \dots, t_n)$ for clarity. Accordingly, the set FOF of first order formulas is the smallest set satisfying:

- $P(t_1, \dots, t_n) \in FOF$ is a formula, referred to as the atomic formula.
- Compound formulas: $(\neg\varphi)$ (negation), $(\varphi \rightarrow \psi)$ (implication), and $(\forall x\varphi)$ (universal quantification) are formulas (thus in FOF), provided that $\varphi, \psi \in FOF$.

We omit parentheses if it is clear from the context.

As syntactic sugar, we can define $\exists x\varphi$ as $\exists x\varphi := \neg\forall x\neg\varphi$. We assume that $\forall$ and $\exists$ have higher precedence than all logical operators.

Examples of first-order logics

1. Mathematical theories:

- **Presburger Arithmetic** $\langle \mathbb{N}, 0, 1, +, =, < \rangle$.
- **Peano Arithmetic** $\langle \mathbb{N}, 0, S, +, \cdot, =, < \rangle$
- **Tarski Algebra** $\langle \mathbb{R}, 0, +, \cdot, =, < \rangle$
- **Group** $\langle e, +, = \rangle$.
- **Equivalence** $\langle R \rangle$.

2. Example

- Write “every son of my father is my brother” in predicate logic.
- Let *me* denote “me”, $S(x, y)$ (x is a son of y), $F(x, y)$ (x is the father of y), and $B(x, y)$ (x is a brother of y) be predicate symbols of arity 2. Consider

$$\forall x \forall y (F(x; \textit{me}) \wedge S(y; x) \rightarrow B(y; \textit{me})).$$

- Alternatively, let f ($f(x)$ is the father of x) be a unary function symbol. Consider

$$\forall x (S(x; f(\textit{me})) \rightarrow B(x; \textit{me})).$$

- **Translating an English sentence into predicate logic can be tricky.**

Definition 3.1.6 (Sub-formulas). *For a formula φ , we define the sub-formula function $Sf : FOF \rightarrow 2^{FOF}$ as follows:*

$$\begin{aligned}
Sf(P(t_1, \dots, t_n)) &= \{P(t_1, \dots, t_n)\} \\
Sf(\neg\varphi) &= \{\neg\varphi\} \cup Sf(\varphi) \\
Sf(\varphi \rightarrow \psi) &= \{\varphi \rightarrow \psi\} \cup Sf(\varphi) \cup Sf(\psi) \\
Sf(\forall x\varphi) &= \{\forall x\varphi\} \cup Sf(\varphi) \\
Sf(\exists x\varphi) &= \{\exists x\varphi\} \cup Sf(\varphi)
\end{aligned}$$

Definition 3.1.7 (Scope). *The part of a logical expression to which a quantifier is applied is called the scope of this quantifier. Formally, each sub-formula of the form $Qx\psi \in Sf(\varphi)$, the scope of the corresponding quantifier Qx is ψ . Here $Q \in \{\forall, \exists\}$.*

Substitution for Terms

Definition 3.1.8 (Sentence). *We say an occurrence of x in φ is **free** if it is not in scope of any quantifiers $\forall x$ (or $\exists x$). Otherwise, we say that this occurrence is a **bound** occurrence. If a variable φ has no free variables, it is called a closed formula, or a sentence.*

Definition 3.1.9 (Substitution). *The **substitution** of x with t within φ , denoted as $S_t^x \varphi$, is obtained from φ by replacing each free occurrence of x with t .*

We would extend this notation to $S_{t_1, \dots, t_n}^{x_1, \dots, x_n} \varphi$.

Remark 3.1.10. *It is important to remark that $S_{t_1, \dots, t_n}^{x_1, \dots, x_n} \varphi$ is not the same as $S_{t_1}^{x_1} \dots S_{t_n}^{x_n} \varphi$: the former performs a **simultaneous** substitution.*

For example, consider the formula $P(x, y)$: the substitution $S_{y, x}^{x, y} P(x, y)$ gives $S_{y, x}^{x, y} P(x, y) = P(y, x)$ while the substitutions $S_y^x S_x^y P(x, y)$ give $S_y^x S_x^y P(x, y) = S_y^x P(x, x) = P(y, y)$.

Remark 3.1.11. *Consider $\varphi = \exists y(x < y)$ in the number theory. What is $S_t^x \varphi$ for the special case of $t = y$?*

Definition 3.1.12 (Substitutable on Terms). *We say that t is **substitutable** for x within φ iff for each variable y occurring in t , there is no free occurrence of x in scope of $\forall y / \exists y$ in φ .*

Definition 3.1.13 (α - β condition). *If the formula φ and the variables x and y fulfill:*

- 1. y has no free occurrence in φ , and*
- 2. y is substitutable for x within φ ,*

then we say that φ , x and y meet the α - β condition, denoted as $C(\varphi, x, y)$.

Lemma 3.1.14. *If $C(\varphi, x, y)$, then $S_x^y S_y^x \varphi = \varphi$.*

3.2 The Axiom System: the Hilbert's System

As for propositional logic, also FOL can be axiomatized.

Definition 3.2.1 (Axioms). 1. $\varphi \rightarrow (\psi \rightarrow \varphi)$

2. $(\varphi \rightarrow (\psi \rightarrow \eta)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \eta))$

3. $(\neg\varphi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \varphi)$

4. $\forall x\varphi \rightarrow S_t^x\varphi$

if t is substitutable for x within φ

5. $\forall x(\varphi \rightarrow \psi) \rightarrow (\forall x\varphi \rightarrow \forall x\psi)$

6. $\varphi \rightarrow \forall x\varphi$

if x is not free in φ

7. $\forall x_1 \dots \forall x_n \varphi$

if φ is an instance of (one of) the above axioms

$$\text{MP Rule: } \frac{\varphi \rightarrow \psi \quad \varphi}{\psi}$$

Definition 3.2.2 (Syntactical Equivalence). *We say φ and ψ are syntactically equivalent iff $\varphi \vdash \psi$ and $\psi \vdash \varphi$.*

Theorem 3.2.3. (*Gen*): *If x has no free occurrence in Γ , then $\Gamma \vdash \varphi$ implies $\Gamma \vdash \forall x\varphi$.*

Solution. Suppose that $\varphi_0, \varphi_1, \dots, \varphi_n = \varphi$ is the deductive sequence of φ from Γ .

- If φ_i is an instance of some axiom, then according to (A7), $\forall x\varphi_i$ is also an axiom.
- If $\varphi_i \in \Gamma$, since x is not free in Γ , we have $\vdash \varphi_i \rightarrow \forall x\varphi_i$ according to (A6). Therefore, we have $\Gamma \vdash \forall x\varphi_i$ in this case.
- If φ_i is obtained by applying (MP) to some φ_j and $\varphi_k = \varphi_j \rightarrow \varphi_i$. By induction, we have $\Gamma \vdash \forall x\varphi_j$ and $\Gamma \vdash \forall x(\varphi_j \rightarrow \varphi_i)$. With (A5) and (MP), we also have $\Gamma \vdash \forall x\varphi_i$ in this case.

Thus, we have $\Gamma \vdash \forall x\varphi_n$, i.e., $\Gamma \vdash \forall x\varphi$.

Exercise 3.2.4. *Prove that*

1. $\forall x \forall y \varphi \vdash \forall y \forall x \varphi$,
2. $\exists x \forall y \varphi \vdash \forall y \exists x \varphi$.

Exercise 3.2.5. *Prove that*

1. $\forall x(\varphi \rightarrow \psi) \vdash \forall x(\neg \psi \rightarrow \neg \varphi)$,
2. $\forall x(\varphi \rightarrow \psi) \vdash \exists x \varphi \rightarrow \exists x \psi$.

Exercise 3.2.6. *Prove that*

1. *If $\Gamma \vdash \varphi$ and $\Gamma \vdash \neg\psi$, then $\Gamma \vdash \neg(\varphi \rightarrow \psi)$,*
2. *$\forall x\neg(\varphi \rightarrow \psi) \vdash \neg(\varphi \rightarrow \exists x\psi)$.*

Lemma 3.2.7. *(Ren): If $C(\varphi, x, y)$, then $\forall x\varphi$ and $\forall yS_y^x\varphi$ are syntactical equivalent. That is,*

1. *$\forall x\varphi \vdash \forall yS_y^x\varphi$.*
2. *$\forall yS_y^x\varphi \vdash \forall x\varphi$.*

Lemma 3.2.8. *(RS): Let η_ψ^φ denote the formula obtained by replacing (some or all) φ inside η by ψ .*

If $\varphi \vdash \psi$ and $\psi \vdash \varphi$ then $\eta \vdash \eta_\psi^\varphi$ and $\eta_\psi^\varphi \vdash \eta$.

Solution. By induction on the structure of η .

Lemma 3.2.9. *If $C(\varphi, x, y)$ and $\Gamma \vdash \psi$, then $\Gamma \vdash \psi_{\forall y S_y^x \varphi}^{\forall x \varphi}$.*

Solution. An immediate result of (Ren) and (RS).

Theorem 3.2.10. (*GenC*) If $\Gamma \vdash S_a^x \varphi$ where a does not occur in $\Gamma \cup \{\varphi\}$, then $\Gamma \vdash \forall x \varphi$.

3.3 Semantics of FOL

To give semantics of terms/formulas of first order logic, we need an appropriate structure in which interpret the functions and predicates of FOL.

Definition 3.3.1. A *Tarski structure* is a pair $\mathcal{J} = \langle \mathcal{D}, \mathcal{I} \rangle$, where:

- $\mathcal{D}$ is a non-empty set, called the *domain*.
- For each n -ary function f , we have $\mathcal{I}(f) \in \mathcal{D}^n \rightarrow \mathcal{D}$.
- For each n -ary predicate P , we have $\mathcal{I}(P) \in \mathcal{D}^n \rightarrow \{0, 1\}$.

Thus, for each constant a , we have $\mathcal{I}(a) \in \mathcal{D}$.

Definition 3.3.2. Given a Tarski structure $\mathcal{J} = \langle \mathcal{D}, \mathcal{I} \rangle$, an *assignment* σ under $\mathcal{J}$ is a mapping $\sigma: VS \rightarrow \mathcal{D}$.

We use $\Sigma_{\mathcal{J}}$ to denote the set consisting of assignments under $\mathcal{J}$.

Definition 3.3.3. Let $\mathcal{J} = \langle \mathcal{D}, \mathcal{I} \rangle$ and $\sigma \in \Sigma_{\mathcal{J}}$.

Each term t is interpreted to an element $\mathcal{J}(t)(\sigma)$ belonging to $\mathcal{D}$:

- If $t = x$ is a variable, then $\mathcal{J}(t)(\sigma) = \sigma(x)$.
- If $t = f(t_1, \dots, t_n)$ where f is an n -ary function, then $\mathcal{J}(t)(\sigma) = \mathcal{I}(f)(\mathcal{J}(t_1)(\sigma), \dots, \mathcal{J}(t_n)(\sigma))$.

Thus, if $t = a$ is a constant, then $\mathcal{J}(t)(\sigma) = \mathcal{I}(a)$.

Definition 3.3.4. Each formula φ has a truth value $\mathcal{J}(\varphi)(\sigma) \in \{0, 1\}$:

- If $\varphi = P(t_1, \dots, t_n)$, where P is an n -ary predicate, then $\mathcal{J}(\varphi)(\sigma) = \mathcal{I}(P)(\mathcal{J}(t_1)(\sigma), \dots, \mathcal{J}(t_n)(\sigma))$.

- If $\varphi = \neg\psi$, then $\mathcal{I}(\varphi)(\sigma) = 1 - \mathcal{I}(\psi)(\sigma)$.
- If $\varphi = \psi \rightarrow \eta$, then

$$\mathcal{I}(\varphi)(\sigma) = \begin{cases} 1 & \text{if } \mathcal{I}(\psi)(\sigma) = 0 \text{ or } \mathcal{I}(\eta)(\sigma) = 1, \\ 0 & \text{if } \mathcal{I}(\psi)(\sigma) = 1 \text{ and } \mathcal{I}(\eta)(\sigma) = 0. \end{cases}$$

- If $\varphi = \forall x\psi$, then

$$\mathcal{I}(\varphi)(\sigma) = \begin{cases} 1 & \text{if } \mathcal{I}(\psi)(\sigma[x/d]) = 1 \text{ for each } d \in \mathcal{D}, \\ 0 & \text{if } \mathcal{I}(\psi)(\sigma[x/d]) = 0 \text{ for some } d \in \mathcal{D} \end{cases}$$

where $\sigma[x/d]$ is a new assignment defined as

$$\sigma[x/d](y) = \begin{cases} \sigma(y) & \text{if } y \neq x, \\ d & \text{if } y = x. \end{cases}$$

We write $(\mathcal{I}, \sigma) \models \varphi$ if $\mathcal{I}(\varphi)(\sigma) = 1$.

Theorem 3.3.5 (Theorem of Substitution). *Suppose that t is substitutable for x within φ , then*

$(\mathcal{I}, \sigma) \models S_t^x \varphi$ if and only if $(\mathcal{I}, \sigma[x/\mathcal{I}(t)(\sigma)]) \models \varphi$.

We say that $\mathcal{I}$ is a **model** of φ , denoted as $\mathcal{I} \models \varphi$, if $(\mathcal{I}, \sigma) \models \varphi$ for each $\sigma \in \Sigma_{\mathcal{I}}$.

In particular, we say that $\mathcal{I} = \langle \mathcal{D}, \mathcal{I} \rangle$ is a **frugal model** of φ if $|\mathcal{D}|$ is not more than the cardinality of the language.

Recall that φ is a **sentence**, if there is no free variable occurring in φ .

Theorem 3.3.6. *If φ is a sentence, then*

- $\mathcal{I} \models \varphi$ iff $(\mathcal{I}, \sigma) \models \varphi$ for **some** $\sigma \in \Sigma_{\mathcal{I}}$.

Definition 3.3.7. *Let φ, ψ be FOL formulas and Γ be a set of FOL formulas. Then we define:*

- $(\mathcal{I}, \sigma) \models \Gamma$ if for each $\eta \in \Gamma$, $(\mathcal{I}, \sigma) \models \eta$;
- $\Gamma \models \varphi$ if for each $\mathcal{I}$ and $\sigma \in \Sigma_{\mathcal{I}}$, $(\mathcal{I}, \sigma) \models \Gamma$ implies $(\mathcal{I}, \sigma) \models \varphi$;
- φ and ψ are equivalent if $\{\varphi\} \models \psi$ and $\{\psi\} \models \varphi$;
- φ is valid if $\emptyset \models \varphi$.

Definition 3.3.8 (Tautology for FOL). *For a formula $\varphi \in FOL$, we construct φ' as follows:*

- *for each sub-formula ψ of φ which is either an atomic formula, or a formula of the form $\forall x\eta$, we replace it with a corresponding propositional variable p_ψ .*

If φ' is a tautology in propositional logic, then we say φ is a tautology for FOL.

Definition 3.3.9 (Prenex Normal Form (PNF)). *A formula is in prenex normal form if and only if it is of the form $Q_1x_1Q_2x_2\dots Q_kx_kP(x_1, x_2, \dots)$ where each $Q_i, i = 1, 2, \dots, k$ is either the existential quantifier or the universal quantifier, and $P(x_1, \dots, x_k)$ is a predicate involving no quantifiers.*

Question: can we transform a formula into an equivalent P-NF form?

3.4 A Sound and Complete Axiomatization for FOL without Equality $\approx$

3.4.1 The Axiom System: Soundness

Similarly to propositional logic, for FOL we have the soundness property:

Theorem 3.4.1. *If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$.*

Hint. For proving the theorem, show and make use of the following results:

- $\{\forall x(\varphi \rightarrow \psi), \forall x\varphi\} \models \forall x\psi$;
- if x is not free in φ , then $\vdash \varphi \rightarrow \forall x\varphi$. $\square$

Corollary 3.4.2. *If $\vdash \varphi$, then $\models \varphi$.*

3.4.2 The Axiom System: Completeness

A **Hintikka set** Γ is a set of FOL formulas fulfilling the following properties:

1. For each atomic formula φ (i.e, $\varphi = P(t_1, \dots, t_n)$, where $n \geq 0$), either $\varphi \notin \Gamma$ or $\neg\varphi \notin \Gamma$.
2. $\varphi \rightarrow \psi \in \Gamma$ implies that either $\neg\varphi \in \Gamma$ or $\psi \in \Gamma$.
3. $\neg\neg\varphi \in \Gamma$ implies that $\varphi \in \Gamma$.
4. $\neg(\varphi \rightarrow \psi) \in \Gamma$ implies that $\varphi \in \Gamma$ and $\neg\psi \in \Gamma$.
5. $\forall x\varphi \in \Gamma$ implies that $S_t^x\varphi \in \Gamma$ for each t which is substitutable for x within φ .
6. $\neg\forall x\varphi \in \Gamma$ implies that there is some t with $C(\varphi, x, t)$ such that $\neg S_t^x\varphi \in \Gamma$.

Note: $C(\varphi, x, t)$ iff $C(\varphi, x, y)$ for all y occurring in t .

Lemma 3.4.3. *A Hintikka set Γ is consistent, and moreover, for each formula φ , either $\varphi \notin \Gamma$, or $\neg\varphi \notin \Gamma$.*

Theorem 3.4.4. *A Hintikka set Γ is satisfiable, i.e, there is some interpretation $\mathcal{I}$ and some $\sigma \in \Sigma_{\mathcal{I}}$ such that $(\mathcal{I}, \sigma) \models \varphi$ for each $\varphi \in \Gamma$.*

Theorem 3.4.5. *If Γ is a set of FOL formulas, then “ Γ is consistent” implies that “ Γ is satisfiable”.*

Particularly, if Γ consists only of sentences, then Γ has a frugal model.

Proof. Let us enumerate¹ the formulas as $\varphi_0, \varphi_1, \dots, \varphi_n, \dots$, and subsequently define a series of formula sets as follows. Let $\Gamma_0 = \Gamma$, and

$$\Gamma_{i+1} = \begin{cases} \Gamma_i \cup \{\neg\varphi_i\} & \text{if } \Gamma_i \vdash \neg\varphi_i \\ \Gamma_i \cup \{\varphi_i\} & \text{if } \Gamma_i \not\vdash \neg\varphi_i \text{ and } \varphi_i \neq \neg\forall x\psi \\ \Gamma_i \cup \{\varphi_i, \neg S_a^x\psi\} & \text{if } \Gamma_i \not\vdash \neg\varphi_i, \text{ and } \varphi_i = \neg\forall x\psi \end{cases}$$

Above, for each formula $\forall x\psi$, we pick and fix the constant a which does not occur in $\Gamma_i \cup \{\varphi_i\}$.

¹We assume the language to be countable, yet the result can be extended to languages with arbitrary cardinality.

Finally let $\Gamma^* = \lim_{i \rightarrow \infty} \Gamma_i$.

If Γ is consistent, the set Γ^* is maximal and consistent, and is referred to as the [Henkin set](#). Thus, a Henkin set is also a Hintikka set. $\square$

Theorem 3.4.6. *If $\Gamma \models \varphi$, then $\Gamma \vdash \varphi$.*

Corollary 3.4.7. *If $\models \varphi$, then $\vdash \varphi$.*

Theorem 3.4.8. *Γ is consistent iff each of its finite subset is consistent. Moreover, Γ is satisfiable iff each of its finite subsets is satisfiable.*

3.5 A Sound and Complete Axiomatization for FOL with Equality $\approx$

The axiomatization based on the Hilbert's systems seen in the previous section can be extended to the case of first order logic with the equality $\approx$. To do this, two additional axioms have to be included in the Hilbert's system:

$A_{\approx}$: $x \approx x$;

$A'_{\approx}$: $(x \approx y) \rightarrow (\alpha \rightarrow \alpha_y^x)$, where α is an atomic formula.

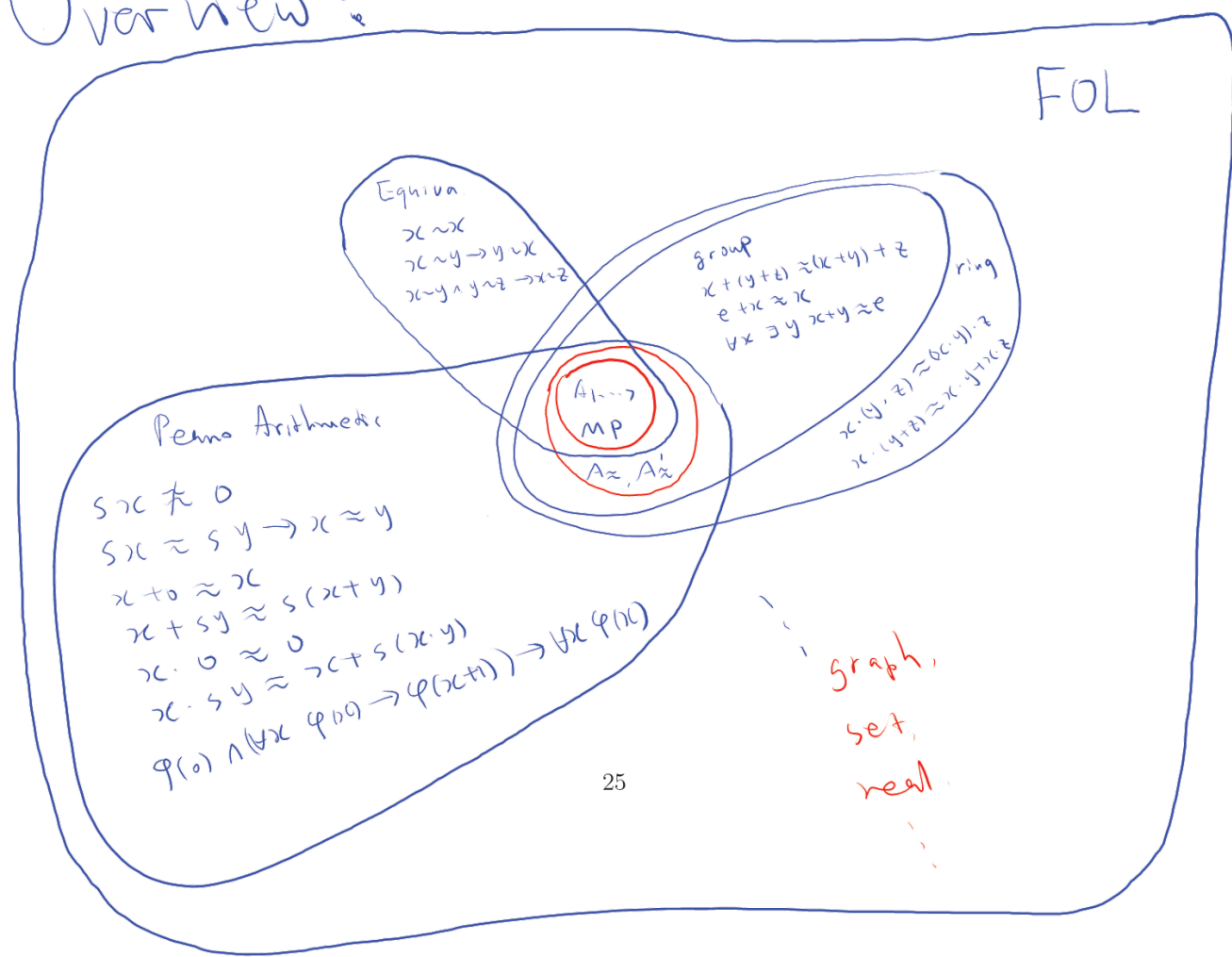
The soundness and completeness results can be proved similarly in the extended Hilbert's system; note that for the completeness one, a variation of the Tarski structure is required, namely, the domain considered in the construction modulo the relation $\approx$. This allows us to manage correctly the formulas that are equivalent under $\approx$.

The actual details about the above construction are omitted; the interested reader is invited to formalize them.

α is atomic formula.

Overview:

FOL



3.6 Exercises

Let A be a finite alphabet, such as the English alphabet, and consider the following simple programming language **WHILE** whose well-formed programs are those obtained by applying the rules of the following grammar:

Numbers:

$$Num ::= d \mid dNum \quad \text{where } d \in \{0, 1, \dots, 9\}$$

Identifiers:

$$\begin{aligned} Id &::= aId' && \text{where } a \in A \\ Id' &::= \lambda \mid aId' \mid NumId' \end{aligned}$$

Numeric expressions:

$$\begin{aligned} Exp &::= Num \mid Id \\ &\mid Exp + Exp \mid Exp - Exp \mid Exp * Exp \mid Exp / Exp \end{aligned}$$

Boolean expressions:

$$\begin{aligned} BExp &::= \mathbf{true} \mid \mathbf{false} \mid Exp = Exp \\ &\mid \neg BExp \mid BExp \wedge BExp \mid BExp \vee BExp \end{aligned}$$

Programs:

$$\begin{aligned} P &::= \mathbf{skip} \mid Id := Exp \mid P; P \\ &\mid \mathbf{if } BExp \mathbf{ then } P \mathbf{ else } P \mathbf{ fi} \\ &\mid \mathbf{while } BExp \mathbf{ do } P \mathbf{ done} \end{aligned}$$

Exercise 3.6.1. *Show, by providing the appropriate functions, that the set of identifiers has the same cardinality as $\mathbb{N}$.*

Exercise 3.6.2. *Show that the set of all well-formed **WHILE** programs is countable.*

Exercise 3.6.3. *Show the following statements:*

1. *If A is an uncountable set and B is a countable set, then $|A \setminus B| = |A|$.*
2. *Let $\mathbb{I}$ be the set of irrational numbers, i.e., $\mathbb{I} = \{ r \in \mathbb{R} \mid r \notin \mathbb{Q} \}$. Then $|\{0, 1\}^\omega \setminus \{0, 1\}^*0^\omega| = |\mathbb{I} \cap [0, 1]|$.*
3. $|\{0, 1\}^\omega| = |[0, 1]|$.
4. $|[0, 1]| = |[0, 1] \times [0, 1]| = |[0, 1]^n|$ for each $n \geq 1$.

Hint. For point 3, make use of points 1 and 2; for point 4, make use of point 3.

Exercise 3.6.4. Consider the semantic bracket operator presented in Definition 2.1.6 of the lecture notes. Show that $\models \varphi \rightarrow \psi$ iff $\llbracket \varphi \rrbracket \subseteq \llbracket \psi \rrbracket$.

Exercise 3.6.5. Let $\mathfrak{Q}_1 \subseteq \text{FOL}$ be the set of FOL formulas φ such that each quantifier operator Qx (with $Q \in \{\forall, \exists\}$ and $x \in \text{VS}$) appears at most once in φ .

Provide a function *Scope*, including its type, such that, given a quantifier operator Qx (with $Q \in \{\forall, \exists\}$ and $x \in \text{VS}$) and a formula $\varphi \in \mathfrak{Q}_1$, it returns the formula corresponding to the scope of Qx .

Exercise 3.6.6. *Given $\varphi, \psi, \eta \in FOL$,*

- 1. provide a function $\#$, including its type, that returns how many times ψ occurs in φ as sub-formula;*
- 2. provide a function R_∞ , including its type, that replaces each occurrence of φ in η with ψ ;*
- 3. provide a function R_1 , including its type, that replaces exactly one occurrence of φ in η with ψ if φ occurs in η , and that returns η if φ does not occur in η . If φ occurs in η multiple times, then there is no requirement on the particular instance to be replaced.*

Exercise 3.6.7. *Prove the following:*

1. For any predicate P with arity 2, $\forall x \forall y P(x, y) \vdash \forall y \forall z P(y, z)$.
2. Assume x is not free in φ , then $\varphi \rightarrow \forall x \psi$ and $\forall x(\varphi \rightarrow \psi)$ are syntactically equivalent.
3. We say a formula φ has repeated occurrences of a bound variable x , if Qx appears more than once in the sub-formulas of φ (recall $Q \in \{\forall, \exists\}$). Prove that there exists a formula φ' which has no repeated occurrences of any bound variable such that φ and φ' are syntactically equivalent.

Exercise 3.6.8. Prove the following:

Q1 $\neg \forall x \varphi \vdash \exists x \neg \varphi$; and $\neg \exists x \varphi \vdash \forall x \neg \varphi$.

Q2 $\forall x \varphi \wedge \psi \vdash \forall x(\varphi \wedge \psi)$, if x does not occur in ψ .

Q3 $\exists x \varphi \vee \exists x \psi \vdash \exists x(\varphi \vee \psi)$.

Exercise 3.6.9. Let S be a binary predicate symbol, P and Q unary predicate symbols. Prove the following:

$$Q1 \quad \exists x \exists y (S(x, y) \vee S(y, x)) \vdash \exists x \exists y S(x, y).$$

$$Q2 \quad \forall x \forall y \forall z (S(x, y) \wedge S(y, z) \rightarrow S(x, z)), \forall x \neg S(x, x) \vdash \forall x \forall y (S(x, y) \rightarrow \neg S(y, x)).$$

$$Q3 \quad \exists x \exists y (S(x, y) \vee S(y, x)), \neg \exists x S(x, x) \vdash \exists x \exists y (x \neq y).$$

$$Q4 \quad \forall x (P(x) \vee Q(x)) \vdash \forall x P(x) \vee \forall x Q(x) \text{ is not provable.}$$

Exercise 3.6.10. Prove Lemma 3.4.3: a Hintikka set Γ is consistent, and moreover, for each formula φ , either $\varphi \notin \Gamma$, or $\neg \varphi \notin \Gamma$.

Exercise 3.6.11. Given a formula φ , let $\mathfrak{H}(\varphi)$ be the set of Hintikka sets containing φ , that is, $\mathfrak{H}(\varphi) = \{ \Gamma \subseteq FOF \mid \varphi \in \Gamma \text{ and } \Gamma \text{ is a Hintikka set} \}$. We say that $\Gamma \in \mathfrak{H}(\varphi)$ is minimal if, for each $\Gamma' \in \mathfrak{H}(\varphi)$, it holds that $\Gamma' \subseteq \Gamma$ implies $\Gamma' = \Gamma$; we denote by $\mathfrak{m}(\varphi)$ the set of minimal Hintikka sets in $\mathfrak{H}(\varphi)$, that is, $\mathfrak{m}(\varphi) = \{ \Gamma \in \mathfrak{H}(\varphi) \mid \Gamma \text{ is minimal} \}$.

1. Provide a minimal Hintikka set $\Gamma_\varphi \in \mathfrak{m}(\varphi)$ for the formula

$$\varphi = \forall x \forall y (\neg(x \approx y) \rightarrow (R(x, y) \rightarrow \neg R(y, x)))$$

under the assumption that $VS = \{x, y\}$, $FS = CS = \{a, b\}$, $PS = \{R\}$, and $ES = \{\approx\}$.

2. Prove that $\mathfrak{H}(\varphi) \cap \mathfrak{H}(\neg\varphi) = \emptyset$ for each $\varphi \in FOF$.
3. Let $PL \subseteq FOF$ be the set of FOL formulas in which each predicate appears at most once and in which no quantifier $Q \in \{\forall, \exists\}$ occurs. Define a function $c: PL \rightarrow \mathbb{N}$ such that, for each $\varphi \in PL$, returns the number of different minimal Hintikka sets containing φ .

Chapter 4

Algorithms

Definition 4.0.12 (Algorithm ([from wikipedia](#))). • *An algorithm is a set of rules that precisely defines a sequence of operations, which can perform calculation, data processing and automated reasoning tasks.*

- *An algorithm is an effective method that can be expressed within a finite amount of space and time and in a well-defined formal language for calculating a function. Starting from an initial state and initial input (perhaps empty), the instructions describe a computation that, when executed, proceeds through a finite number of well-defined successive states, eventually producing “output” and terminating at a final ending state. The transition from one state to the next is not necessarily deterministic; some algorithms, known as randomized algorithms, incorporate random input.*
- *History*
 - *The concept of algorithm has existed for centuries;*
 - *what would become the modern algorithm began with attempts to solve the Entscheidungsproblem (the “dec-*

sion problem”) posed by David Hilbert in 1928.

- Subsequent formalizations were framed as attempts to define “effective calculability” or “effective method”; those formalizations included the Gödel-Herbrand-Kleene recursive functions, Alonzo Church’s lambda calculus, Emil Post’s “Formulation 1”, and Alan Turing’s Turing machines.
- Giving a formal definition of algorithms, corresponding to the intuitive notion, remains a challenging problem.
- Church-Turing Thesis: any real-world computation can be translated into an equivalent computation involving a Turing machine.
- An algorithm can be considered to be any sequence of operations that can be simulated by a Turing-complete system.

Definition 4.0.13 (Algorithm (Cont'd)). • *Expressing algorithms: high-level description, implementation-level description and formal description.*

- *Complexity analysis: **Formal versus empirical, execution efficiency**.*
- *Classification*
 - *By implementation: recursion, logical, serial, parallel, distributed, deterministic vs nondeterministic, exact vs approximation, quantum*
 - *By design paradigm: brute-force or exhaustive search, divide and conquer, search and enumeration, randomized algorithms.*
 - *By optimization problems: linear programming, dynamic programming, integer programming, semi-definite programming, the greedy method, the heuristic method*
 - *By field of study*
 - *By complexity: space complexity and time complexity.*

4.1 Syntax of an algorithm in pseudo-code

In this part, we consider an extension of the programming language **WHILE** presented in the Exercise sheet 2. Basically, the grammar is extended with *arrays* and *procedures*.

Let A be a finite alphabet, such as the English alphabet, and consider the following simple programming language **PROC**, an extension of **WHILE**, whose well-formed programs are those obtained by applying the rules of the following grammar:

Numbers:

$$Num ::= d \mid dNum \quad \text{where } d \in \{0, 1, \dots, 9\}$$

Identifiers:

$$\begin{aligned} Id &::= aId' && \text{where } a \in A \\ Id' &::= \lambda \mid aId' \mid NumId' \end{aligned}$$

Numeric expressions:

$$\begin{aligned} Exp &::= Num \mid Id \mid Id[Exp] \mid \mathbf{length}(Id) \mid (Exp) \\ &\mid Exp + Exp \mid Exp - Exp \mid Exp * Exp \mid Exp / Exp \\ &\mid Id(Exp, \dots, Exp) \end{aligned}$$

Boolean expressions:

$$\begin{aligned} BExp &::= \mathbf{true} \mid \mathbf{false} \mid Exp = Exp \mid (BExp) \\ &\mid Exp \leq Exp \mid Exp < Exp \\ &\mid Exp \geq Exp \mid Exp > Exp \\ &\mid \neg BExp \mid BExp \wedge BExp \mid BExp \vee BExp \end{aligned}$$

Procedure declaration

$$\begin{aligned} Pr ::= & \text{procedure } Id(Id, \dots, Id) \ P \\ & | \text{procedure } Id(Id, \dots, Id) \ P; \text{return } Exp \end{aligned}$$

Programs:

$$\begin{aligned} P ::= & \text{skip} \mid Id := Exp \mid Id[Exp] := Exp \mid P; P \\ & \mid \text{if } BExp \text{ then } P \text{ else } P \text{ fi} \\ & \mid \text{while } BExp \text{ do } P \text{ done} \\ & \mid \text{for } Id := Exp \text{ to } Exp \text{ do } P \text{ done} \\ & \mid Id(Exp, \dots, Exp) \end{aligned}$$

4.2 Examples of algorithms

Algorithm 4.2.1 (Collatz Conjecture). The following pseudo-code in **PROC** terminates when the value of n becomes to 1. The input n is a natural number, greater than or equal to 1.

```
procedure Collatz( $n$ )
  while  $n \neq 1$  do
    if even( $n$ ) then
       $n := n/2$ 
    else
       $n := n \times 3 + 1$ 
    fi
  done
```

Discussion: **Does this algorithm terminate?**

Algorithm 4.2.2 (Maximum in a generic array). The following pseudo-code in **PROC** returns the maximum value occurring in the provided **array**.

```
procedure max(array)
  maxvalue := array[0];
  for i := 1 to length(array) - 1 do
    if array[i] > maxvalue then
      maxvalue := array[i]
    else
      skip
    fi
  done;
  return maxvalue
```

Algorithm 4.2.3 (Index of a value in an array). The following pseudo-code in **PROC** returns the index of the specified **value** if it occurs in the provided **array**, otherwise **length(array)** is returned.

```
procedure index0f(value, array)
  index := length(array);
  for i := 0 to length(array) - 1 do
    if array[i] = value then
      index := i
    else
      skip
    fi
  done;
  return index
```

Algorithm 4.2.4 (Index of a value in a sorted array). The

following pseudo-code in **PROC** returns the index of the specified value if it occurs in the provided sorted **array**, otherwise **length(array)** is returned.

```

procedure indexOfSorted(value, array)
    index := length(array);
    low := 0;
    high := length(array) - 1;
    while low < high do
        middle := (low + high)/2;
        if array[middle] < value then
            low := middle + 1
        else
            high := middle
        fi
    done;
    if array[low] = value then
        index := low
    else
        skip
    fi;
    return index

```

Algorithm 4.2.5 (Swap elements in an array). The following pseudo-code in **PROC** swaps the elements at index **i** and **j** in the provided **array**.

```

procedure swap(array, i, j)
    temp := array[i];
    array[i] := array[j];
    array[j] := temp

```

Algorithm 4.2.6 (Bubble sort). The following pseudo-code in **PROC** sorts the provided **array**.

```
procedure bubbleSort(array)
  for i := 0 to length(array) - 1 do
    for j := 0 to (length(array) - 1) - i do
      if array[j] > array[j + 1] then
        swap(array, j, j + 1)
      else
        skip
      fi
    done
  done
```

Algorithm 4.2.7 (Gnome sort). The following pseudo-code in **PROC** sorts the provided **array**.

```
procedure gnomeSort(array)
  i := 0;
  while i < length(array) do
    if i = 0  $\vee$  array[i - 1]  $\leq$  array[i] then
      i := i + 1
    else
      swap(array, i, i - 1);
      i := i - 1
    fi
  done
```

Algorithm 4.2.8 (Insertion sort). The following pseudo-code in **PROC** sorts the provided **array**.

```

procedure insertionSort(array)
  for i := 1 to length(array) - 1 do
    j := i;
    while j > 0  $\wedge$  array[j - 1] > array[j] do
      swap(array, j, j - 1);
      j := j - 1
    done
  done

```

Greedy Algorithms

Algorithm 4.2.9 (Change Making). The algorithm makes changes $c_1 > c_2 \dots > c_r$ for n cents.

```

procedure procedureChange( $c_1, c_2, \dots, c_r$ )
  for i := 1 to r do
     $d_i := 0$ ;
    while  $n \geq c_i$  do
       $d_i := d_i + 1$ ;
       $n := n - c_i$ 
    done
  done

```

Algorithm 4.2.10 (Earliest Ending Time Job Scheduling). Two jobs are *compatible* if they do not overlap. Assumption: sorts talks with ending times $e_1 < e_2 < \dots e_n$.

```

procedure schedule( $e_1, e_2, \dots, e_n$ )
   $S := \emptyset$ ;
  for i := 1 to n do
    if talk i is compatible with S then  $S := S \cup \{talk\ i\}$ 
  done

```

*Remark: set not defined in **PROC***

Definition 4.2.11. *The Tableau approach for the satisfiability problem of propositional logic: a propositional formula is satisfiable iff it has a consistent tableau.*

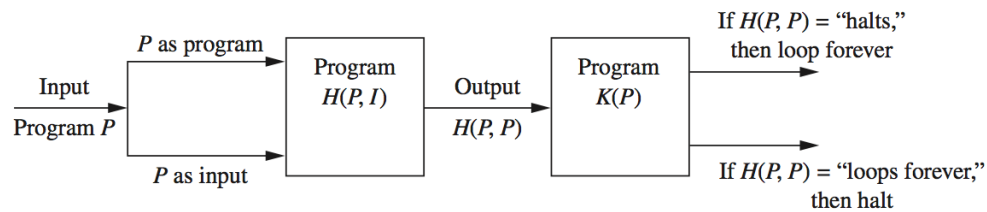
Definition 4.2.12 (Decidability). • *A decision problem consists of a set of instances and a subset of yes-instances.*

- *The Primality problem: is the instance x a prime number?*
- *The answer (solution) to any decision problem is just one bit (true or false).*
- *A problem Q is decidable iff there is an algorithm A , such that for each instance q of Q , the computation $A(q)$ stops with an answer.*
- *A problem Q is semi-decidable iff there is an algorithm A , such that for each instance q of Q , if q holds, then $A(q)$ stops with the positive answer; otherwise, $A(q)$ either stops with the negative answer, or does not stop.*

Definition 4.2.13 (The Halting Problem). *It takes as input a computer program and input to the program and determines whether the program will eventually stop when run with this input.*

- *If the program halts, we have our answer.*
- *If it is still running after any fixed length of time has elapsed, we do not know whether it will never halt or we just did not wait long enough for it to terminate.*

Definition 4.2.14 (Undecidability of the Halting Problem).



4.3 Satisfiability Problem and DPLL

The slides are downloadable from <http://www.computational-logic.org/iccl/master/lectures/summer07/sat/slides/dpll.pdf>

4.4 The Growth of Functions

Definition 4.4.1. *Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that $|f(x)| \leq C|g(x)|$ whenever $x > k$.*

Definition 4.4.2. Let f and g be functions from the set of integers or the set of real numbers to the set of real numbers. We say that $f(x)$ is $\Omega(g(x))$ if there are positive constants C and k such that $|f(x)| \geq C|g(x)|$ whenever $x > k$.

Moreover, We say that $f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ and $\Omega(g(x))$.

Exercise 4.4.3. Show that:

1. Show that $\log n!$ is $O(n \log n)$.
2. Show that n^2 is not $O(n)$.
3. Suppose that $f_1(x)$ is $O(g_1(x))$ and that $f_2(x)$ is $O(g_2(x))$. Then $(f_1 + f_2)(x)$ is $O(\max(|g_1(x)|, |g_2(x)|))$.
4. Suppose that $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$. Then $(f_1 f_2)(x)$ is $O(g_1(x) g_2(x))$.
5. Show that $3x^2 + 8x \log x$ is $\Theta(x^2)$.
6. Assume $a_n \neq 0$. Show that $\sum_{i=0}^n a_i x^i$ is $\Theta(x^n)$.

4.5 Complexity of Algorithms

We are interested in the *time complexity* of the algorithm.

TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

<i>Complexity</i>	<i>Terminology</i>
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Definition 4.5.1. • A problem that is solvable using an algorithm with polynomial worst-case complexity is called *tractable*. Tractable problems are said to belong to class *P*.

- *Problems for which a solution can be checked in polynomial time are said to belong to the class NP .*
- *The $P = NP$ problem asks whether NP , the class of problems for which it is possible to check solutions in polynomial time, equals P , the class of tractable problems.*
- *NP -complete problems: It is an NP problem and if a polynomial time algorithm for solving it were known, then $P = NP$.*
- *The satisfiability problem is NP -complete. Cook-Levin Theorem*

A prize of 1,000,000 dollars is offered by the Clay Mathematics Institute for its solution.

4.6 Logic and Computer Science – Logical Revolution

NOTE: The following material is from Moshe Vardi

Definition 4.6.1 (Hilberts Program). *Hilberts Program (1922-1930): Formalize mathematics and establish that:*

- *Mathematics is consistent:*
a mathematical statement and its negation cannot ever both be proved.
- *Mathematics is complete:*
all true mathematical statements can be proved.
- *Mathematics is decidable: there is a mechanical way to determine whether a given mathematical statement is true or false.*

Definition 4.6.2 (The Demise of Hilberts Program). *Gödel:*

- *Incompleteness of ordinary arithmetic - There is no systematic way of resolving all mathematical questions.*
- *Impossibility of proving consistency of mathematics*
Gödel (1930): "This sentence is not provable."
- *Church and Turing (1936): Unsolvability of first-order logic:*
The set of valid first-order sentences is not computable.

Definition 4.6.3 (Entscheidungsproblem). *Entscheidungsproblem (The Decision Problem) [Hilbert-Ackermann, 1928]: Decide if a given first-order sentence is valid (dually, satisfiable).*

Church-Turing Theorem, 1936: The Decision Problem is unsolvable.

Turing, 1936:

- *Defined computability in terms of Turing machines (TMs)*
- *Proved that the termination problem for TMs is unsolvable ("this machine terminates iff it does not terminate")*
- *Reduced termination to Entscheidungsproblem.*

Definition 4.6.4 (Mathematical Logic - 1936). *Logic as Foundations of Mathematics:*

- *Incomplete (example: Continuum Hypothesis)*
 - *Cannot prove its own consistency*
 - *Unsolvable decision problem*
 - *Unsolvable termination problem*
- Can we get some positive results?*
- *Focus on special cases!*

Definition 4.6.5 (The Fragment-Classification Project). *Idea: Identify decidable fragments of first-order logic - (1915-1983)*

- *Monadic Class (monadic predicates)*
- *Bernays-Schönfinkel Class ($\exists^*\forall^*$)*
- *Ackermann Class ($\exists^*\forall\exists^*$)*
- *Gödel Class ($\exists^*\forall\forall\exists^*$)*

Outcome: Very weak classes! What good is first-order logic?

Definition 4.6.6 (Monadic Logic). *Monadic Class: First-order logic with monadic predicates - captures syllogisms.*

$$\forall x(Man(x) \rightarrow Mortal(x))$$

Löwenheim, 1915: The Monadic Class is decidable.

- *Proof: Bounded-model property - if a sentence is satisfiable, it is satisfiable in a structure of bounded size.*
- *Proof technique: quantifier elimination.*

Definition 4.6.7 (Logic of Integer Addition). *Integer Addition:*

- *Domain: N (natural numbers)*
- *Predicate: =*

- *Addition function:* $+$
- $y = 2x : y = x + x$
- $x \leq y : (\exists z)(y = x + z)$
- $x = 0 : (\forall y)(x \leq y)$
- $x = 1 : x \neq 0 \wedge (\forall y)(y = 0 \vee x \leq y)$
- $y = x + 1 : (\exists z)(z = 1 \wedge y = x + z)$

Bottom Line: Theory of Integer Addition can express Integer Programming (integer inequalities) and much more.

Definition 4.6.8 (Presburger Arithmetics). *Mojzesz Presburger, 1929:*

- *Sound and complete axiomatization of integer addition*
- *Decidability: There exists an algorithm that decides whether a given first-order sentence in integer-addition theory is true or false.*
 - *Decidability is shown using quantifier elimination, supplemented by reasoning about arithmetical congruences.*

- *Decidability can also be shown using automata-theoretic techniques.*

Definition 4.6.9 (Complexity of Presburger Arithmetics). *Complexity Bounds:*

- *Oppen, 1978: $\text{TIME}(2^{2^{\text{poly}}})$ upper bound*
- *Fischer & Rabin, 1974: $\text{TIME}(2^{2^{\text{lin}}})$ lower bound*

Rabin, 1974: "Theoretical Impediments to Artificial Intelligence": "the complexity results point to a need for a careful examination of the goals and methods in AI".

Definition 4.6.10 (Finite Words - Nondeterministic Finite Automata). $A = (\Sigma, S, S_0, \rho, F)$

- *Alphabet:* Σ
- *States:* S
- *Initial states:* $S_0 \subseteq S$
- *Nondeterministic transition function:* $\rho : S \times \Sigma \rightarrow 2^S$
- *Accepting states:* $F \subseteq S$

Definition 4.6.11. *Finite Words - Nondeterministic Finite Automata*

Input word: $a_0, a_1, \dots, a_{n-1}$

Run: $s_0, s_1, \dots, s_n$

- $s_0 \in S_0$
- $s_{i+1} \in \rho(s_i, a_i)$ for $i \geq 0$

Acceptance: $s_n \in F$

Recognition: $L(A)$ - words accepted by A .

Fact: NFAs define the class *Reg* of regular languages.

Definition 4.6.12 (Logic of Finite Words). *View finite word $w = a_0, \dots, a_{n-1}$ over alphabet Σ as a mathematical structure:*

- *Domain:* $0, \dots, n - 1$
- *Dyadic predicate:* $\leq$
- *Monadic predicates:* $\{P_a : a \in \Sigma\}$

Monadic Second-Order Logic (MSO):

- *Monadic atomic formulas:* $P_a(x)$ ($a \in \Sigma$)
- *Dyadic atomic formulas:* $x < y$
- *Set quantifiers:* $\exists P, \forall P$

Example: $(\exists x)((\forall y)(\neg(x < y)) \wedge P_a(x))$ - last letter is a .

Definition 4.6.13 (Automata and Logic). [*Büchi, Elgot, Trakhtenbrot, 1957-8 (independently)*]: $MSO \equiv NFA$

Both MSO and NFA define the class Reg.

Proof: Effective

- *From NFA to MSO ($A \rightarrow \varphi_A$)*
- *From MSO to NFA ($\varphi \rightarrow A_\varphi$)*

Definition 4.6.14 (NFA Nonemptiness). *Nonemptiness: $L(A) = \emptyset$*

Nonemptiness Problem: Decide if given A is nonempty.

Directed Graph $G_A = (S, E)$ of NFA $A = (\Sigma, S, S_0, \rho, F)$:

- *Nodes: S*
- *Edges: $E = \{(s, t) : t \in \rho(s, a) \text{ for some } a \in \Sigma\}$*

It holds: A is nonempty iff there is a path in G_A from S_0 to F .

Decidable in time linear in size of A , using breadth-first search or depth-first search.

Definition 4.6.15 (MSO Satisfiability - Finite Words). *Satisfiability: $\text{models}(\psi) = \emptyset$*

Satisfiability Problem: Decide if given ψ is satisfiable.

It holds: ψ is satisfiable iff A_ψ is nonempty.

It holds: MSO satisfiability is decidable.

- *Translate ψ to A_ψ .*
- *Check nonemptiness of A_ψ .*

Definition 4.6.16 (Complexity Barrier). *Computational Complexity:*

- *Naive Upper Bound: Nonelementary Growth 2 to the power of the tower of height $O(n)$*

- *Lower Bound [Stockmeyer, 1974]: Satisfiability of FO over finite words is nonelementary (no bounded-height tower).*

Definition 4.6.17 (Program Verification). • *The Dream - Hoare, 1969: "When the correctness of a program, its compiler, and the hardware of the computer have all been established with mathematical certainty, it will be possible to place great reliance on the results of the program."*

- *The Nightmare - De Millo, Lipton, and Perlis, 1979: "We believe that . . . program verification is bound to fail. We cannot see how it is going to be able to affect anyone's confidence about programs."*
- *"... software verification ... has been the Holy Grail of computer science for many decades but not in some very key areas, for example, driver verification we are building tools that can do actual proof about the software and how it works in order to guarantee the reliability." (Bill Gates, keynote address at Winhec 2002)*
- *Hoare has pose a grand challenge: "The verification challenge is to achieve a significant body of verified programs that have precise external specifications, complete internal*

specifications, machine-checked proofs of correctness with respect to a sound theory of programming.”

- *NICTA seL4 using Isabelle/HOL, INRIA CompCert using Coq*

The Hoare Triple $\{\varphi\}P\{\psi\}$

Definition 4.6.18 (Logic in Computer Science: c. 1980). *Status: Logic in CS is not too useful!*

- *First-order logic is undecidable.*
- *The decidable fragments are either too weak or too intractable.*
- *Even Boolean logic is intractable.*
- *Program verification is hopeless.*

Definition 4.6.19. *Post 1980: From Irrelevance to Relevance*
A Logical Revolution:

- *Relational databases*
- *Boolean reasoning*
- *Model checking*
- *Termination checking*
- *...*

Definition 4.6.20 (The Temporal Logic of Programs). *CruX:* Need to specify ongoing behavior rather than input/output relation! "Temporal logic to the rescue" [Pnueli, 1977]:

- *Linear temporal logic (LTL) as a logic for the specification of non-terminating programs*
- *Model checking via reduction to MSO*

But: nonelementary complexity!

In 1996, Pnueli received the Turing Award for seminal work introducing temporal logic into computing science and for outstanding contributions to program and systems verification.

Definition 4.6.21 (Examples). • *always not (CS1 and CS2): safety*

- *always (Request implies eventually Grant): liveness*
- *always (Request implies (Request until Grant)): liveness*

Definition 4.6.22 (Model Checking). *”Algorithmic verification” [Clarke & Emerson, 1981, Queille & Sifakis, 1982]: Model checking programs of size m wrt CTL formulas of size n can be done in time mn .*

Linear-Time Response [Lichtenstein & Pnueli, 1985]: Model checking programs of size m wrt LTL formulas of size n can be done in time $m2^{O(n)}$ (tableau heuristics).

Seemingly:

- *Automata: non-elementary*
- *Tableaux: exponential*

Definition 4.6.23 (Back to Automata). *Exponential-Compilation Theorem [Vardi & Wolper, 1983-1986]: Given an LTL formula φ of size n , one can construct an automaton A_φ of size $2^{O(n)}$ such that a trace σ satisfies φ if and only if σ is accepted by A_φ . Automata-Theoretic Algorithms:*

- *LTL Satisfiability: φ is satisfiable iff $L(A_\varphi) \neq \emptyset$ (PSPACE)*
- *LTL Model Checking: $M \models \varphi$ iff $L(M \times A_{\neg\varphi}) = \emptyset$ ($m2^{O(n)}$)*

Today: Widespread industrial usage

Industrial Languages: PSL, SVA (IEEE standards)

Definition 4.6.24 (Solving the Unsolvables). *B. Cook, A. Podelski, and A. Rybalchenko, 2011: "in contrast to popular belief, proving termination is not always impossible"*

- *The Terminator tool can prove termination or divergence of many Microsoft programs.*
- *Tool is not guaranteed to terminate! Explanation:*
- *Most real-life programs, if they terminate, do so for rather simple reasons.*
- *Programmers almost never conceive of very deep and sophisticated reasons for termination.*

Definition 4.6.25 (Logic: from failure to success). *Key Lessons:*

- *Algorithms*
- *Heuristics*
- *Experimentation*
- *Tools and systems*

Key Insight: Do not be scared of worst-case complexity.

- *It barks, but it does not necessarily bite!*

